Wyrd III GUI Tutorial
Doing Everything in the Interface

For Wyrd III 0.1 (Yggdrasil III suite) *October 2026*

Contents

1 Introduction **3**

1.1 What you need . . . 3

2 Starting Wyrd III **4**

3 Creating a Project **4**

3.1 The folder and the sources . . . 4

3.2 New Project . . . 5

3.3 Add the design . . . 5

3.4 Edit wyrd.toml and save . . . 7

4 Project Settings **9**

5 Running the Flow **10**

6 Results **12**

6.1 The run tree and the Summary . . . 12

6.2 All runs . . . 12

6.3 Log nodes and the Messages dock . . . 13

7 Comparing Two Projects **15**

8 Simulation with Bifrost III **16**

9 The Floorplan **19**

9.1 Build a VPR with graphics . . . 19

9.2 Point Wyrd III at it . . . 19

9.3 Open the floorplan . . . 19

9.4 Step through it . . . 20

10 The Sample Project and Midgard III **23**

10.1 The sample project 23
10.2 Midgard III (Verilog Town) 25

11 Where to Go Next **32**

1 Introduction

This tutorial walks through Wyrd III with the mouse. You open the graphical interface, create a project, describe your machine, run a design through Odin III and VPR, read the results and the messages, compare two versions of the design, simulate it cycle by cycle, look at the sample project, and play a level of Midgard III. Every step of Wyrd III itself is done in the interface and has a numbered figure. The command-line tutorial (`wyrd-tutorial.pdf`) covers the same tool from the terminal.

Doing this tutorial, the reader will learn about:

- Starting the interface and finding the project tree, the tabs and the Messages and Flow docks
- Creating a project, adding a design and editing `wyrd.toml` with live validation
- Setting the machine settings (tools, memory budget, VPR for the floorplan) in the Settings dialog
- Running the flow, reading the Flow log, the run tree, the results table and the Messages dock
- Comparing two projects
- Simulating a design with Bifrost III and viewing its waveform
- The sample project and Midgard III (Verilog Town): placing sensors, editing the Verilog, starting a level

Every picture comes from a real run on one machine: a script (`docs/tutorial-gui/shots/make_gui_shots.py`) drives the real windows with Qt's offscreen platform and grabs each widget, so the tutorial can be regenerated after a change. The Midgard III pictures are grabs of the game's own drawing surface. The numbers are those of the real runs. In the pictures the project folder is a scratch folder; yours will have the name you choose.

Conventions. Menu entries are written Menu1 > Menu2 > Item. Buttons and fields are in sans-serif, like this. File and folder housekeeping (making folders, copying files, editing Verilog in an editor) is done in a terminal, and those commands are shown after a `$` prompt; everything Wyrd III does is done in the interface.

1.1 What you need

Wyrd III runs on Linux, including WSL on Windows (WSLg shows the windows). You need Python 3.11 or later, a built Odin III 1.6.0 (the programs `odin3` and `odin3-flow`), a VTR checkout for `vpr` and the architecture files, and Wyrd III installed with its interface, simulation and game extras:

```
$ python3 -m venv .venv
$ .venv/bin/pip install -e ".[gui,sim,midgard]"
```

The machine settings go in `~/.config/wyrd/machine.toml`. If the file does not exist, `wyrd config init` writes one with the tools it finds, and Section 4 shows how to check and change the values in the interface. The design used here is the small `counteradd` project of the command-line tutorial: a 4-bit counter plus a 4-bit adder with a step input. Its Verilog sources are in `docs/tutorial/project/src` of the Wyrd III repository.

2 Starting Wyrd III

1. Start the interface with `wyrd-gui` (it takes an optional project folder, and without one it reopens the last project).

```
$ wyrd-gui
```

2. The main window opens (Figure 1). The Project dock on the left will show the designs, recipes and architectures of a project. The centre has four tabs: Source, Report, wyrd.toml and Simulate; they are grey until a project is open. The bottom has two tabbed docks, Messages and Flow. The View menu shows or hides the docks.

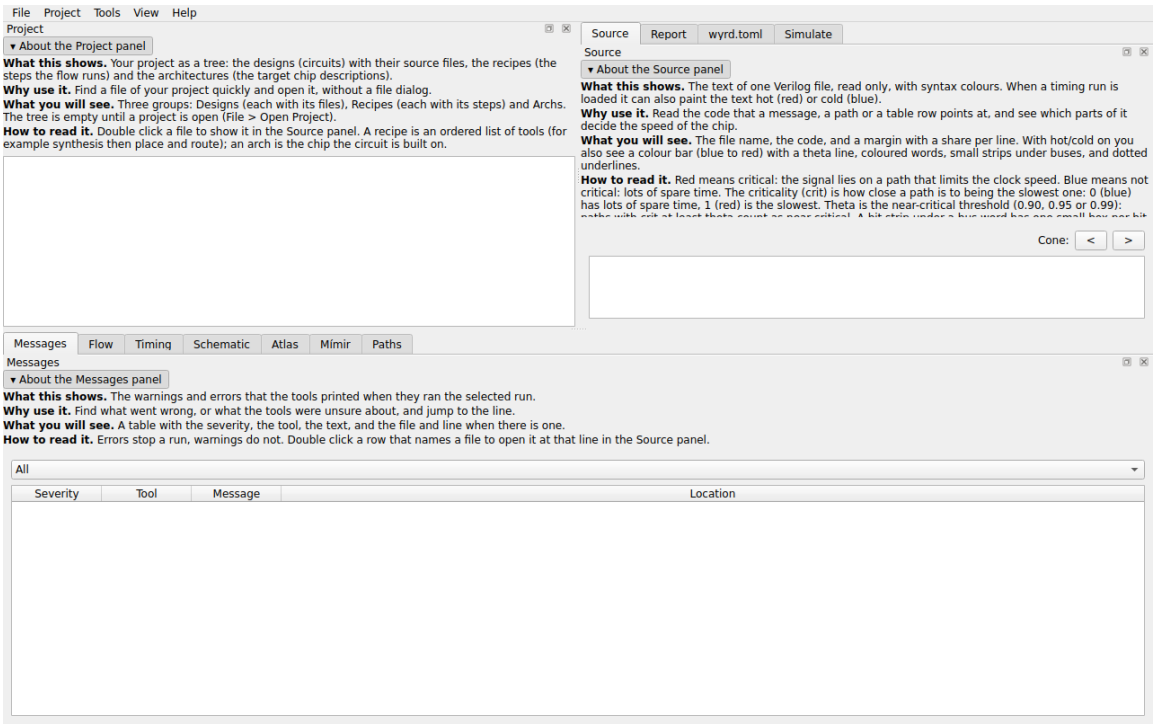


Figure 1. The Wyrd III main window with no project open.

3 Creating a Project

3.1 The folder and the sources

Wyrd III creates the project file, but the folder and the Verilog files come from you. In a terminal, make a folder and copy the three source files into a `src` folder in it:

```
$ mkdir -p ~/wyrd-gui-demo
$ cp -r docs/tutorial/project/src ~/wyrd-gui-demo/src
```

3.2 New Project

1. Open the File menu (Figure 2). New Project... asks for a folder; Open Project... (Ctrl+O) opens an existing one; Recent lists the last projects.

2. Choose File > New Project... and pick `wyrd-gui-demo`. Wyrd III writes a commented `wyrd.toml` into it and opens it. The template names a placeholder file, `CHANGE_ME.v`, so it does not validate yet. The wyrd.toml tab shows the line that is wrong in red and the message under the text (Figure 3); only this tab is enabled until the file is valid.

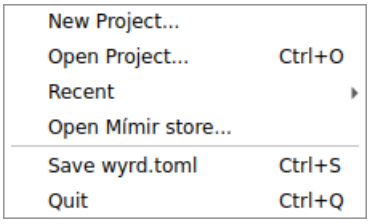


Figure 2. The File menu.

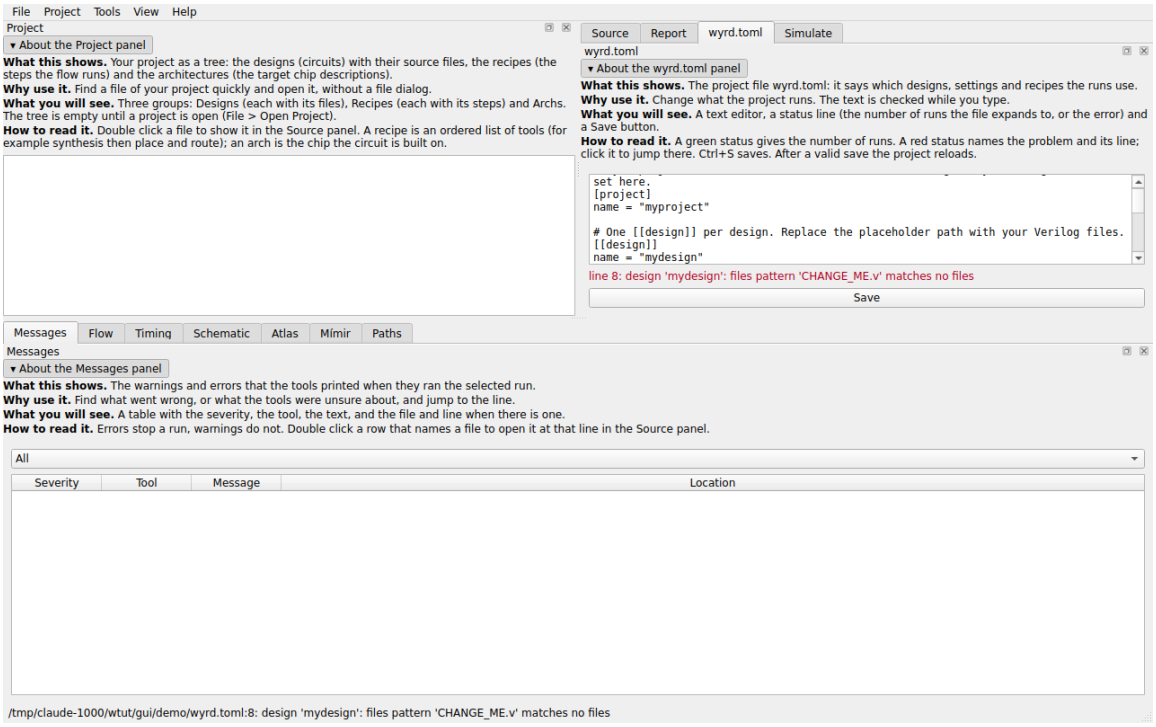


Figure 3. The new project: the template does not validate until the placeholder design is replaced.

3.3 Add the design

1. Open Project > Add design... (Figure 4). The dialog has a name, a list of files, the top module and the box Add to [sweep] design.

2. Name the design `counteradd`, add `src/add4.v`, `src/counter.v` and `src/top.v` with Add files..., set the top module to `top` and tick the box (Figure 5). Press OK.

3. The dialog appends a `[[design]]` block to the text in the wyrd.toml tab and adds the name to the sweep. Nothing is saved yet (Figure 6).

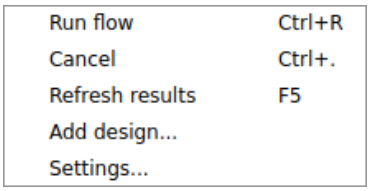


Figure 4. The Project menu.

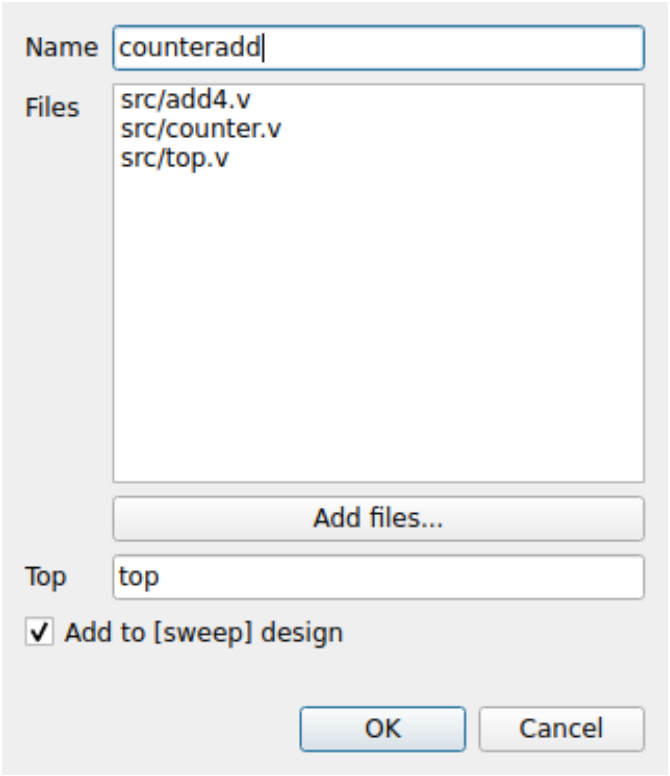


Figure 5. The Add design dialog, filled in.

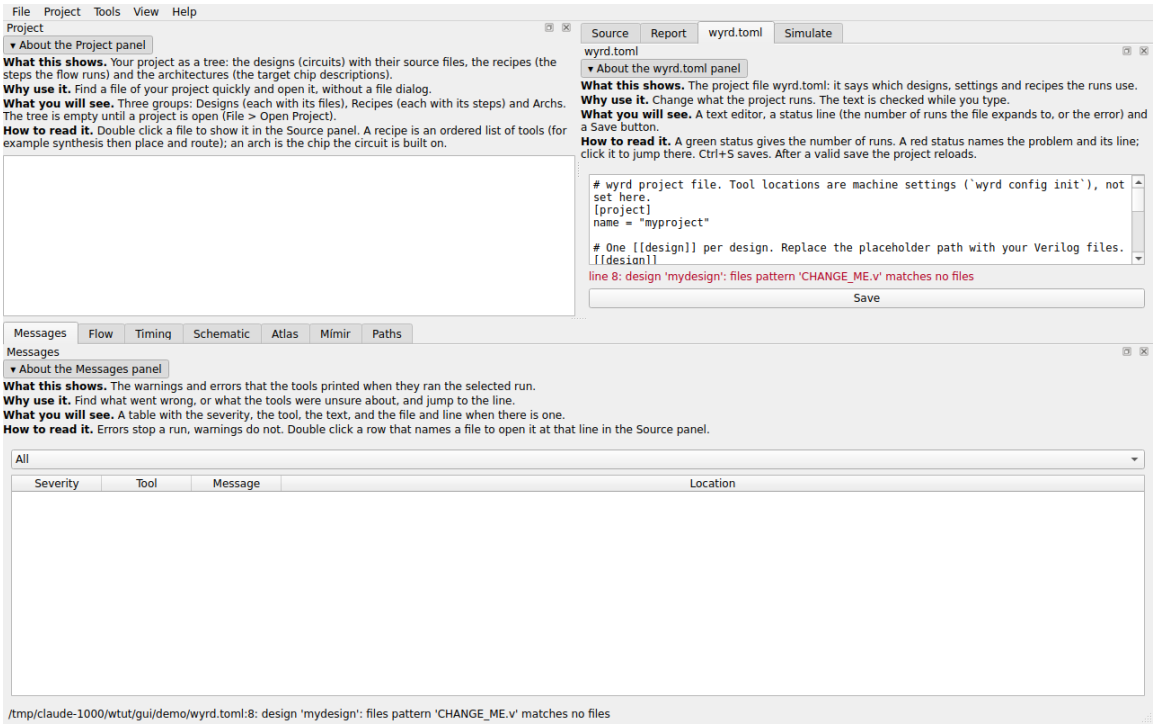


Figure 6. The new design is in the editor text. The placeholder design is still there and the file is still invalid.

3.4 Edit wyrd.toml and save

The wyrd.toml tab is an ordinary text editor with a validation line under it: a valid file shows how many runs it expands to. Edit the text in the tab: delete the placeholder `[[design]]` block and its comment, change the project name to `counteradd`, and add a second architecture, `"EArch"`, to the `arch` list of `[sweep]`. The line under the text turns green and says Valid: 2 runs (Figure 7). Press Save (Ctrl+S). The project reloads: the tree shows the design with its three files, the recipe and the two architectures, and the other tabs are enabled. Double-click a file in the tree to see it in the Source tab (Figure 8).

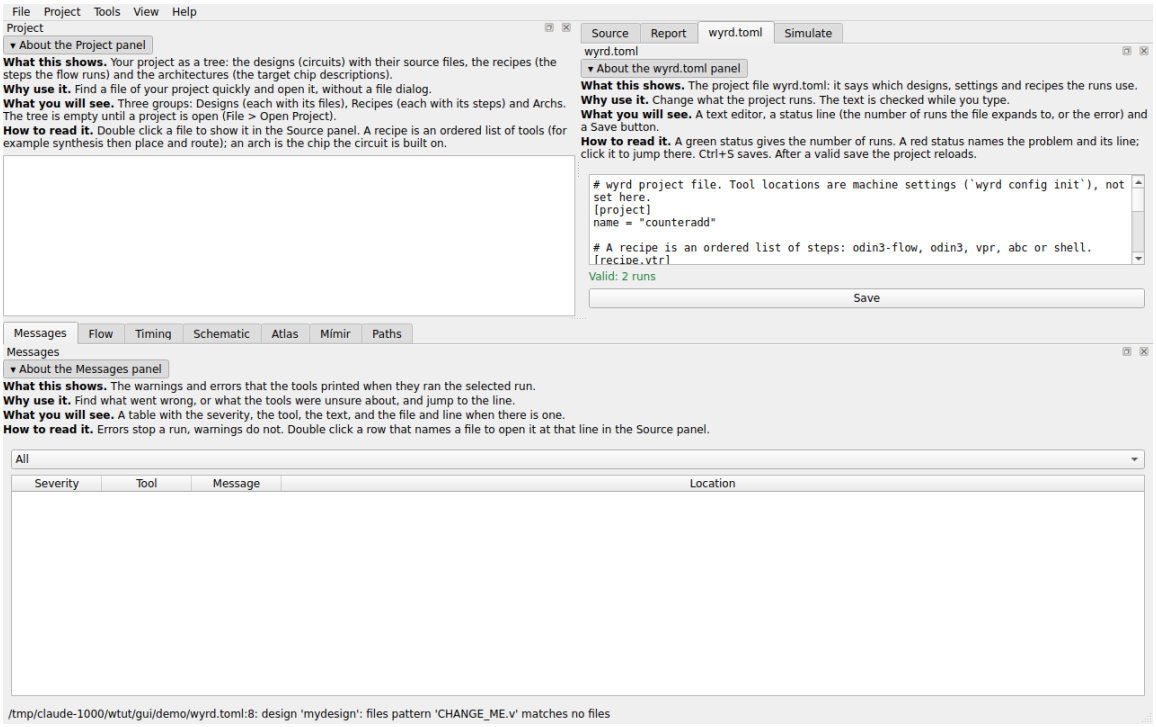


Figure 7. The edited `wyrd.toml`: valid, two runs. The Add design block is at the end of the text.

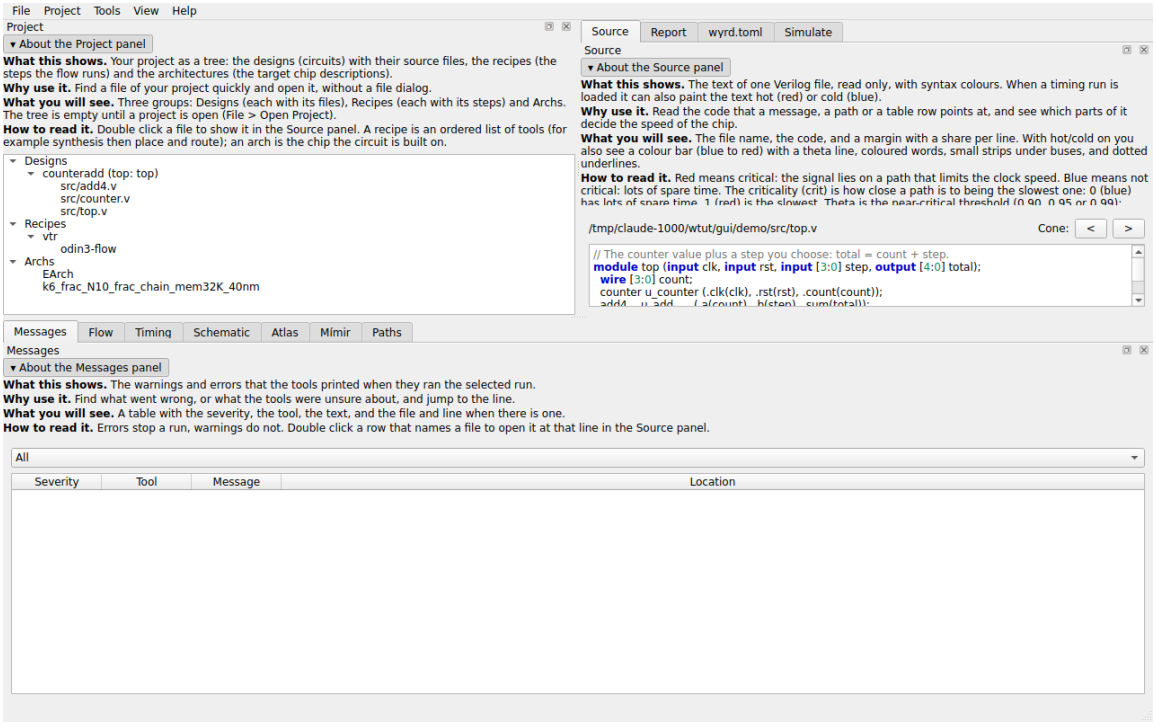


Figure 8. The Source tab with `src/top.v`. The tree on the left shows the saved project.

4 Project Settings

Machine settings describe your computer, not the experiment, so they are not in `wyrd.toml`. Choose Project > Settings... (Figure 9). The top line is the file being edited. The dialog has three groups:

- tools: the `odin3` and `odin3-flow` programs, vtr_root (the VTR checkout, for `vpr`, `abc` and the architecture files) and vpr_gui (a VPR built with graphics; the Floorplan button of Section 9 uses it).
- resources: mem_budget (the share of memory Wyrd III may use), jobs (parallel runs), launcher (`systemd` runs each step in a memory-limited scope) and the free-disk check.
- defaults: the memory a VPR or other step is first given, the factor by which a remembered peak is raised, the factor applied after a memory kill, and the step timeout in seconds.

Press OK to save or Cancel; a value that does not parse is reported in the dialog and the file is not written.

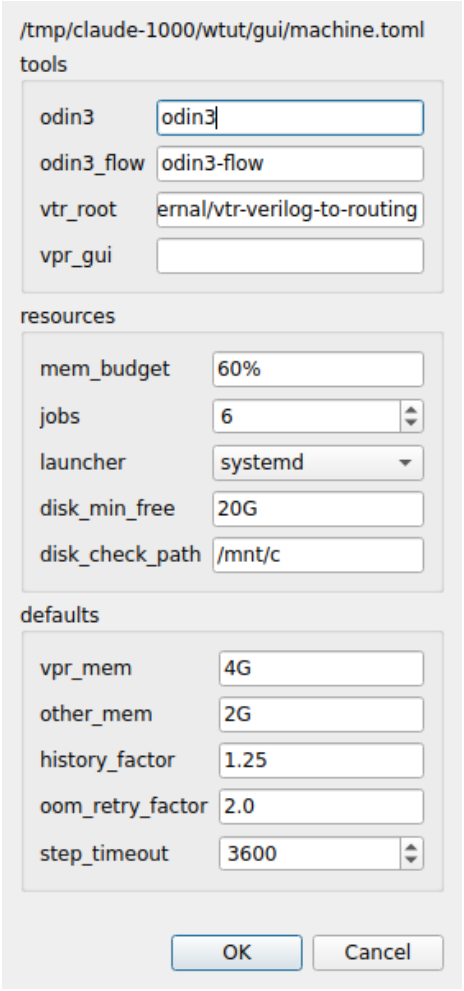


Figure 9. The Settings dialog. The values are those of the machine this tutorial was made on.

5 Running the Flow

1. Click the Flow dock (Figure 10). Jobs and Memory budget override the machine settings for this run when set; Force runs again the runs that are already `ok`.

2. Press Run (or Project > Run flow, Ctrl+R). The bar counts finished runs and the log lists each run as it ends (Figure 11). Cancel (Ctrl+.) stops the running steps; finished runs stay.

3. When the bar reaches 100% the status line reads all runs ok (Figure 12). The Report tab is filled in.

The Flow dock runs `wyrd run` as a program, so what you see is what the command line does. Both runs of this project take about a second each.

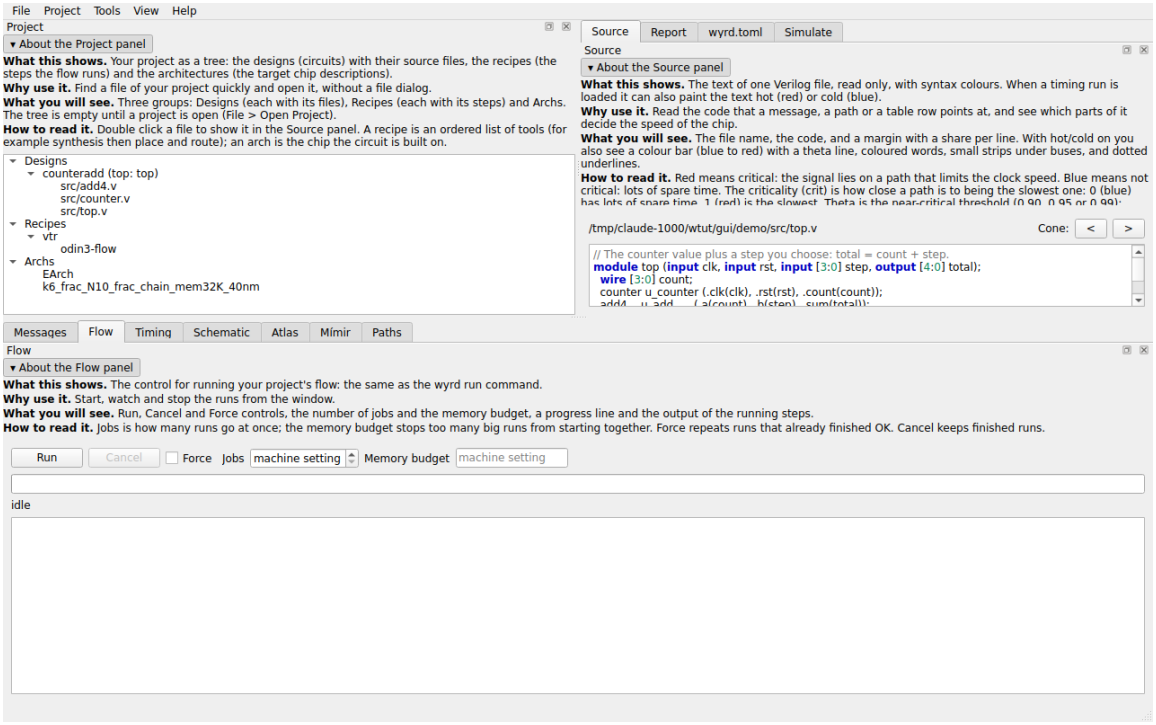


Figure 10. The Flow dock before the first run.

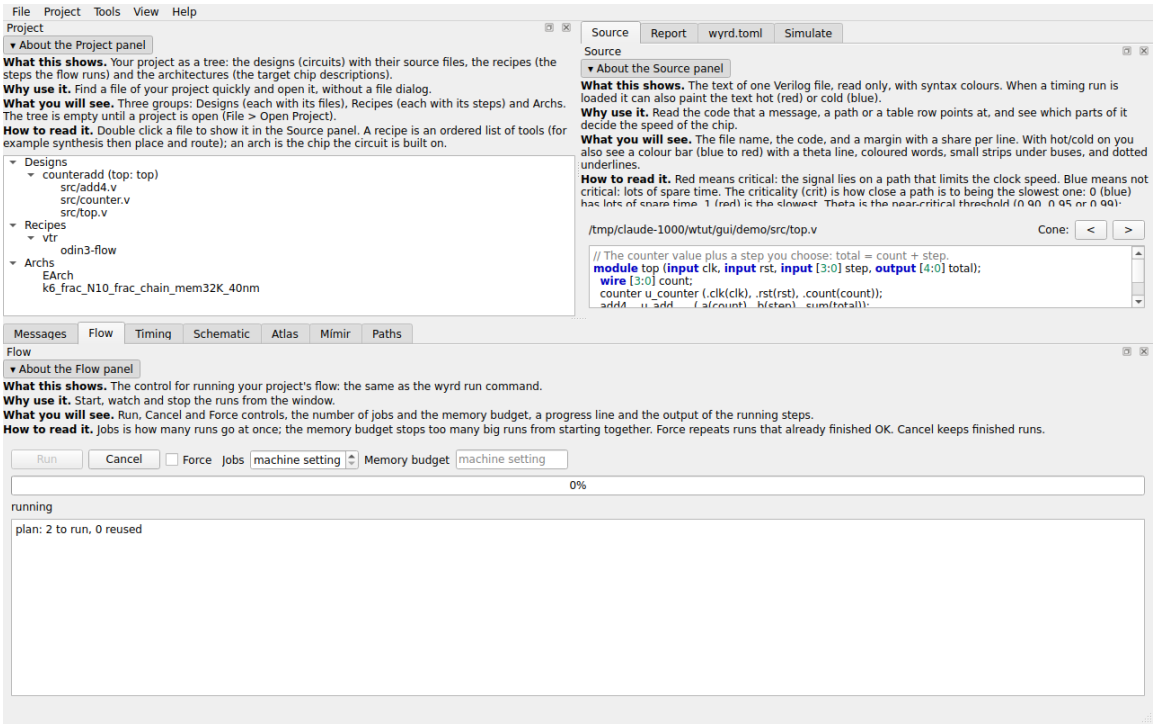


Figure 11. A run in progress.

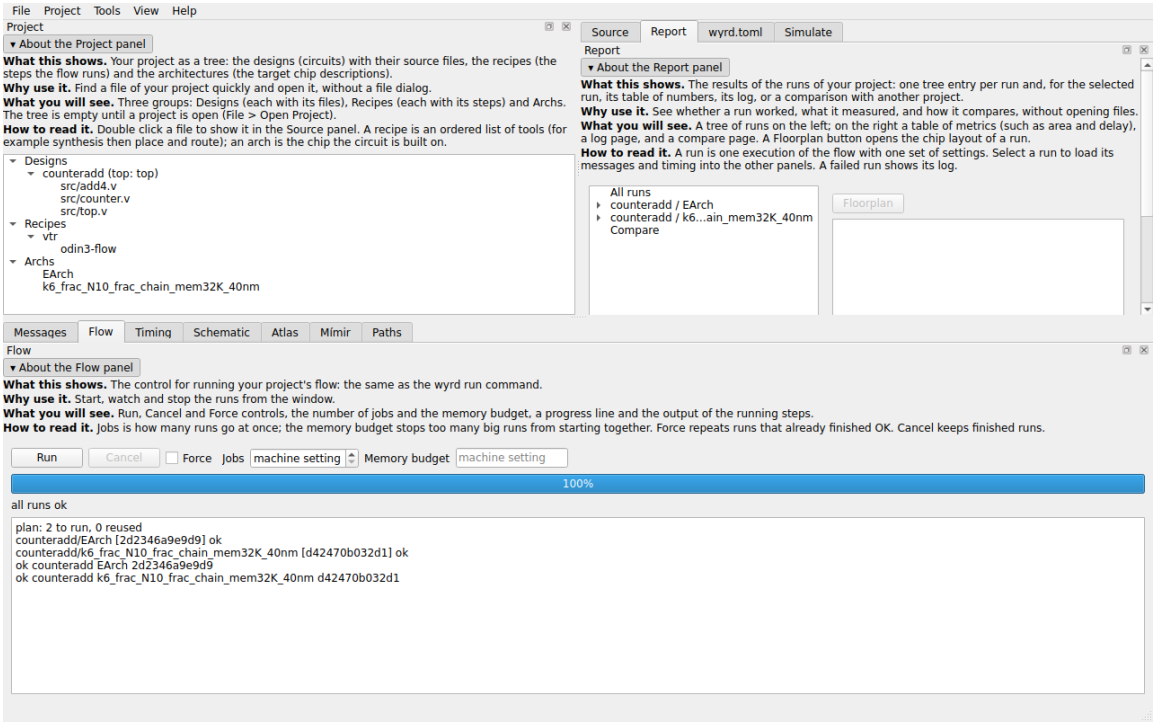


Figure 12. Both runs finished. The Report tab lists one node for each run.

6 Results

6.1 The run tree and the Summary

The tree in the Report tab has All runs first, one node for each run (design and architecture) and Compare last. Expand counteradd / EArch and select Summary (Figure 13): the status, the LUTs and CLBs, the hard blocks (memory and mult_36), Fmax in MHz (1000 divided by the critical path), the minimum channel width, the critical path in ns, the run directory and the run key. For the EArch run: 4 LUTs, 2 CLBs, a critical path of 2.565 ns (Fmax 389.8 MHz) and a channel width of 20.

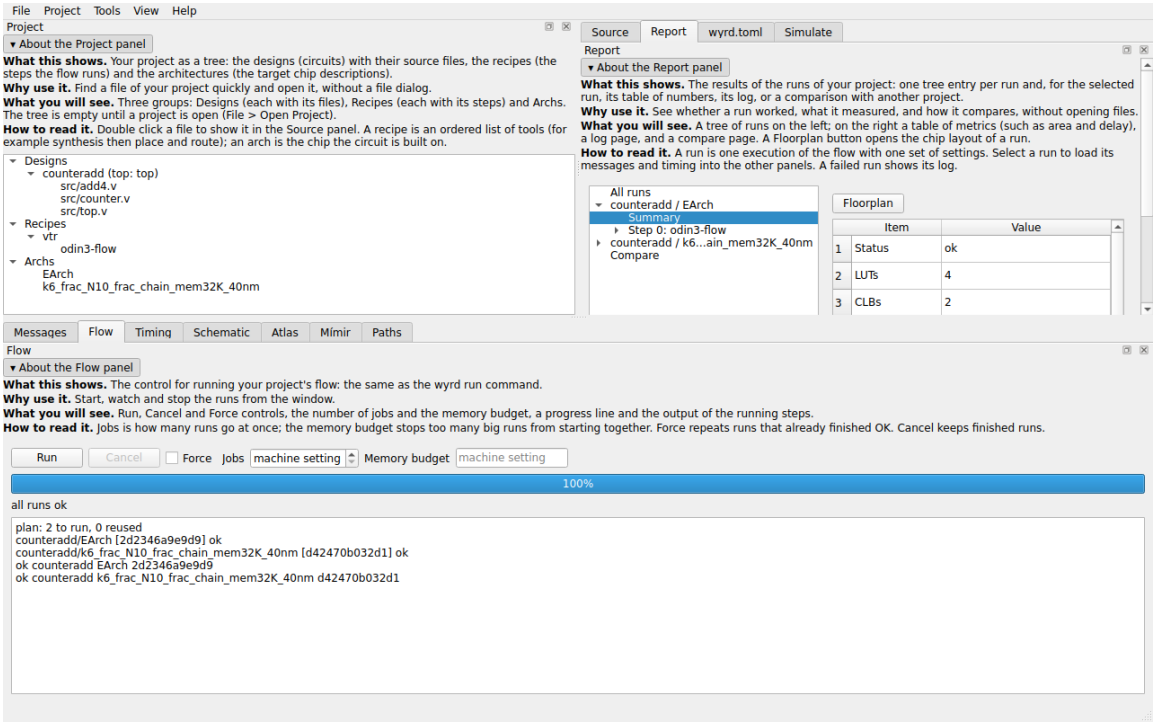


Figure 13. The Summary of the EArch run.

6.2 All runs

Select All runs (Figure 14) for the table that `wyrd table` prints, one row for each run. Here both architectures give 4 LUTs and 2 CLBs; the critical path is 2.565 ns on EArch and 1.984 ns on the K6 architecture.

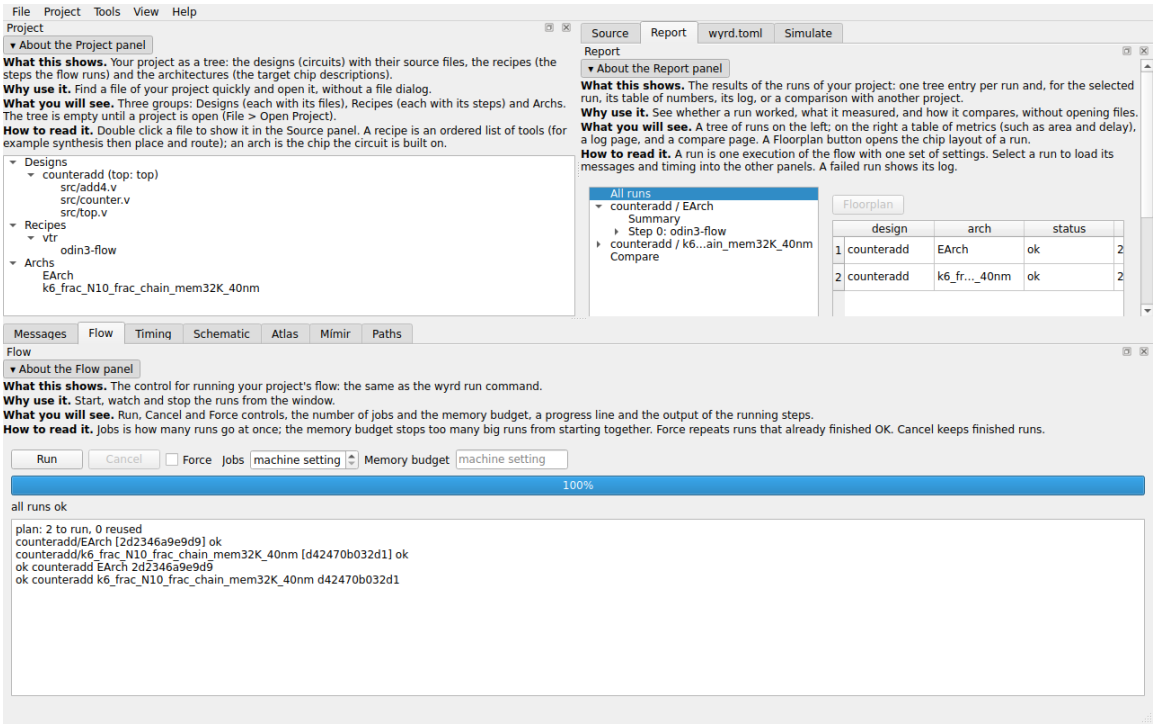


Figure 14. All runs of the project in one table.

6.3 Log nodes and the Messages dock

Each step has a node with its log files. Expand Step 0: odin3-flow and select a log: the page shows the file (Figure 15). Switch to the Messages dock: it lists the warnings and errors that Wyrd III parsed from the logs of the selected run, with the tool that wrote them, the text and the file and line.

1. Use the drop-down at the top of the dock to show All, Errors or Warnings. Here five warnings come from VPR.

2. Double-click a row. The Source tab opens the file at that line, with the line highlighted (Figure 16).

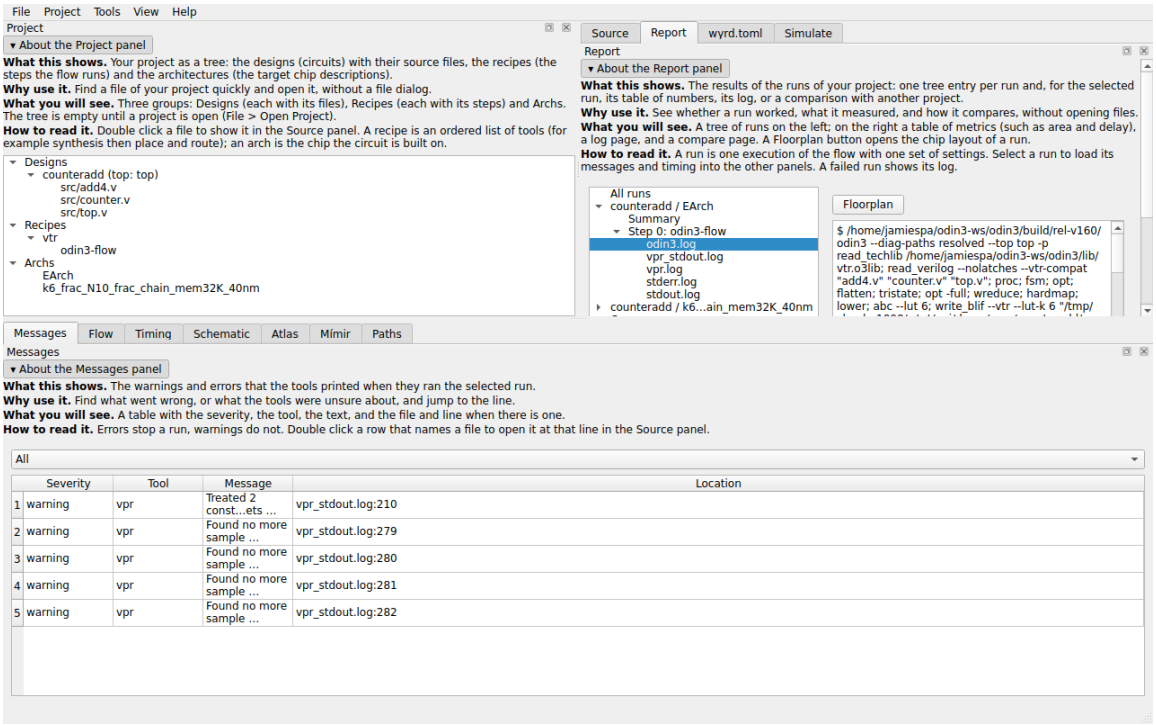


Figure 15. A log node of the first step.

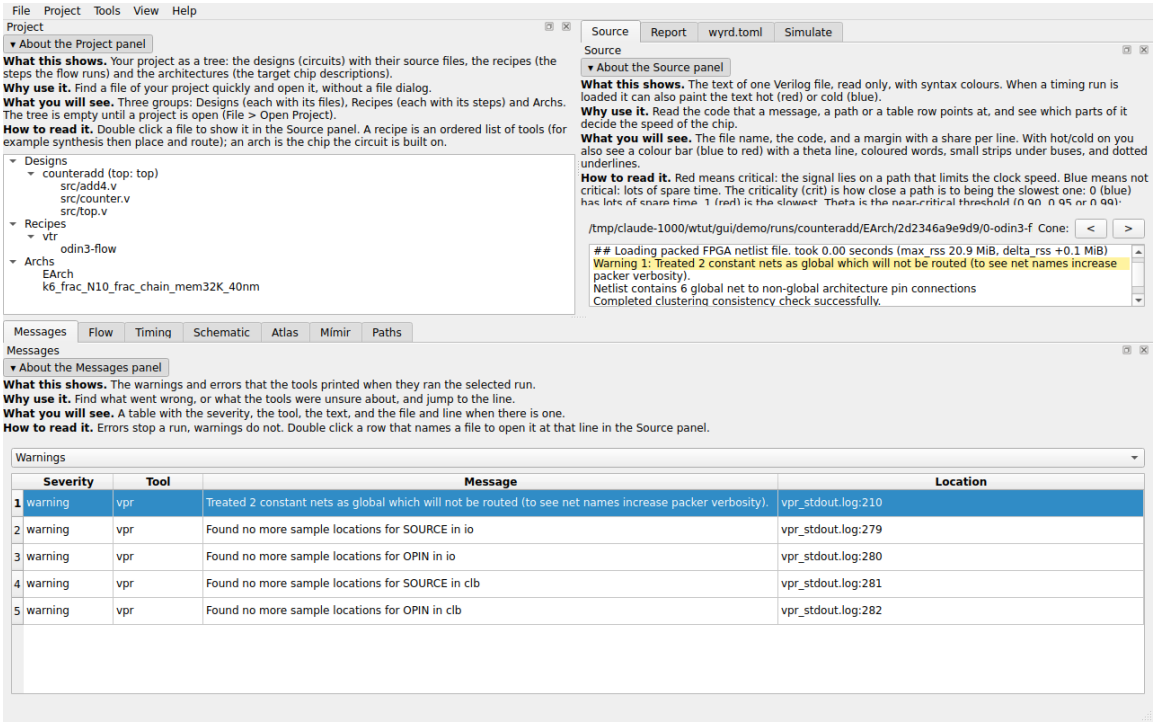


Figure 16. The Messages dock filtered to warnings; a double-click opened the VPR log at the warning.

7 Comparing Two Projects

We compare the design with a variant whose output is registered. Make a copy of the project folder without its results and put the registered `top.v` into it:

```
$ mkdir -p ~/wyrd-gui-demo2
$ cp -r ~/wyrd-gui-demo/src ~/wyrd-gui-demo/wyrd.toml ~/wyrd-gui-demo2/
$ cp docs/tutorial/project/variants/top_registered.v ~/wyrd-gui-demo2/src/top.v
```

1. Choose File > Open Project... and open `wyrd-gui-demo2`, then press Run in the Flow dock.
2. In the Report tab select Compare (Figure 17) and press Compare with project.... Choose `wyrd-gui-demo`.
3. The table lists, for every run of both projects, each metric of the open project (A) and the chosen one (B) and the ratio B/A (Figure 18). The registered design has the shorter critical path on EArch: 2.078 ns against 2.565 ns, a ratio of 1.234; LUTs, CLBs and channel width are unchanged. Runs found in only one project are listed under the table.

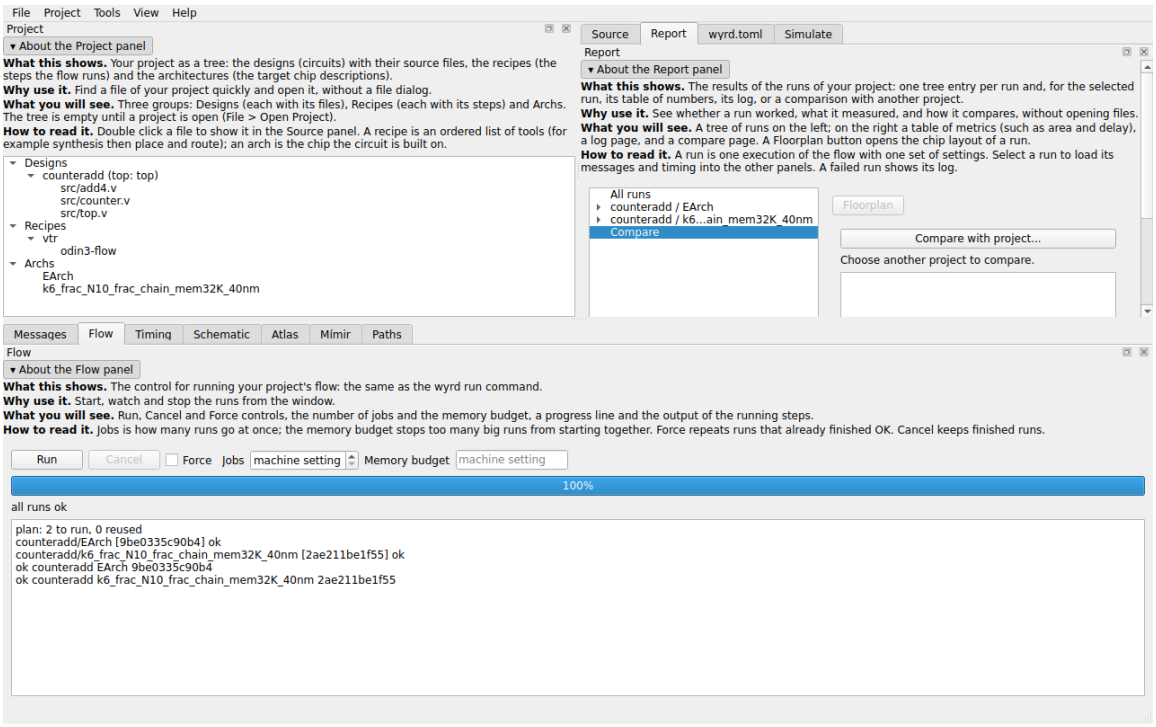


Figure 17. The Compare page of the project with the registered output.

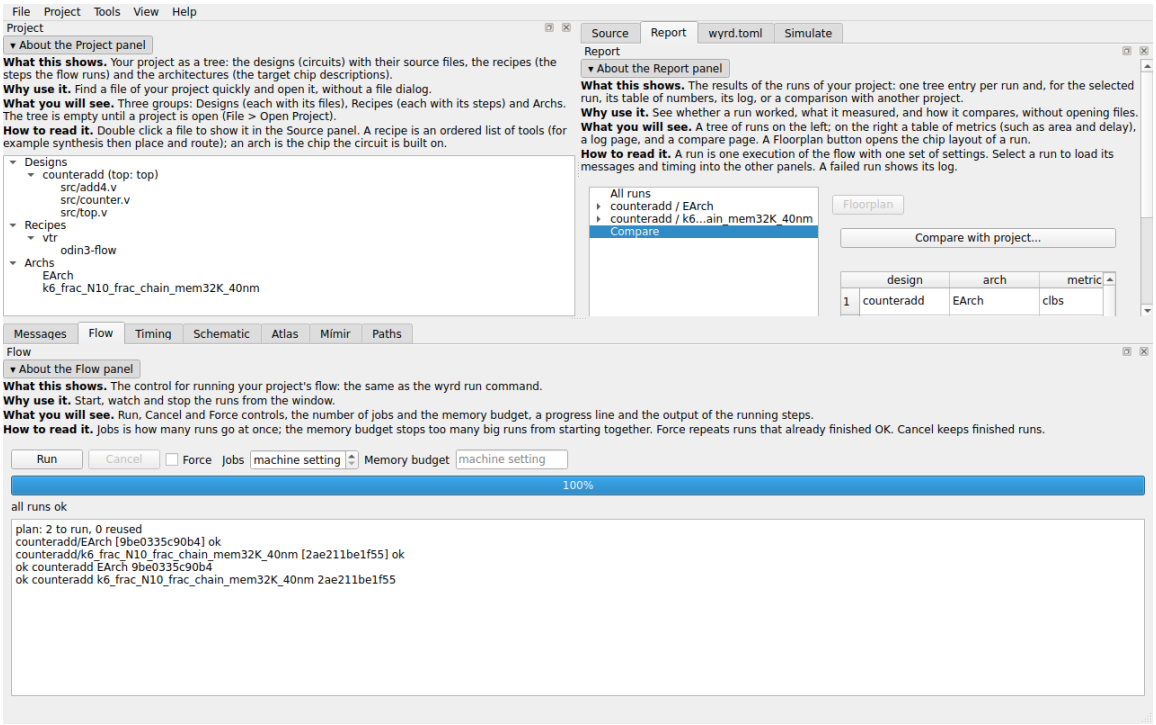


Figure 18. The comparison with the first project.

8 Simulation with Bifrost III

The Simulate tab (Tools menu, Ctrl+T) runs the design cycle by cycle with Odin III's simulator. Re-open `wyrd-gui-demo` and select the tab (Figure 19).

1. Choose the design and, if the design has more than one top, the top module. Press Compile. The panel lists the signals (Figure 20): name, width and kind (input, output or internal).

2. Type values into the input cells: `rst` = 1 and press Step for one clock cycle, then `rst` = 0, `step` = 9 and Step N for five cycles. The output `total` counts up by the step each cycle (Figure 21).

3. Change `step` to 3 and run ten more cycles. The Waveform tab draws every signal with its values (Figure 22): the reset pulse, the step input and `total` in hexadecimal.

4. Run, Reset, Run random and Run vectors are the other ways to drive the design. Save VCD... writes the waveform, Save vectors... the stimulus, and Open in GTKWave opens the VCD if GTKWave is installed.

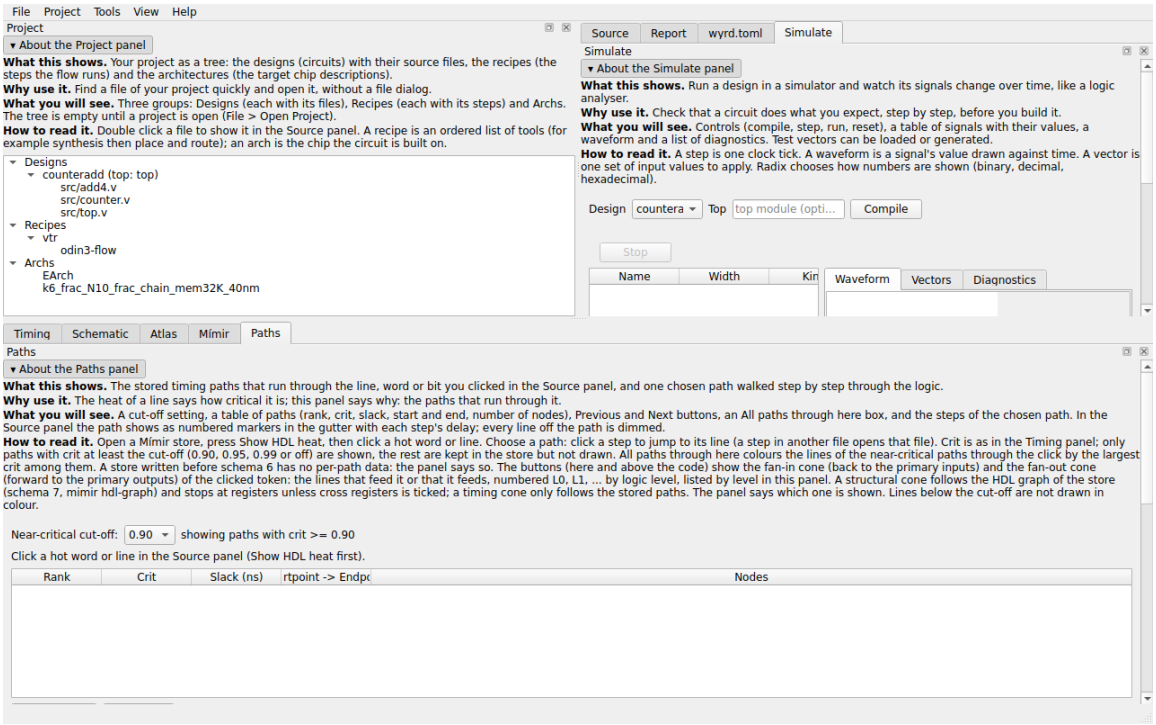


Figure 19. The Simulate tab before compiling.

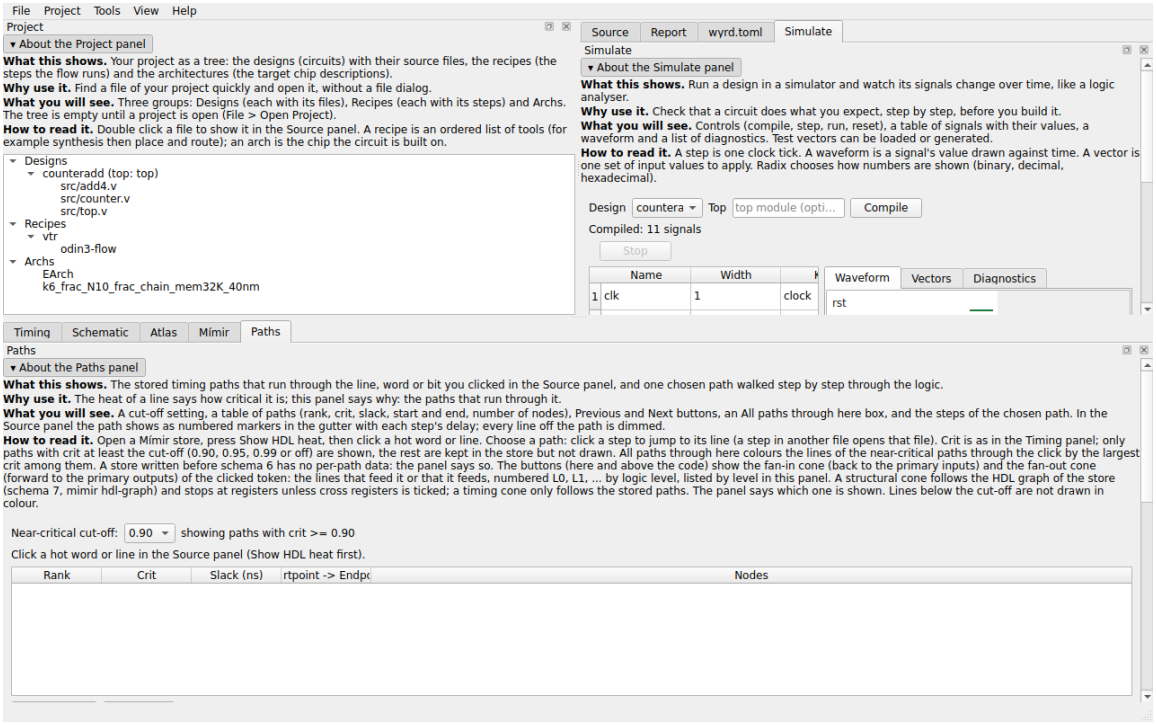


Figure 20. Compiled: the signals of the design.

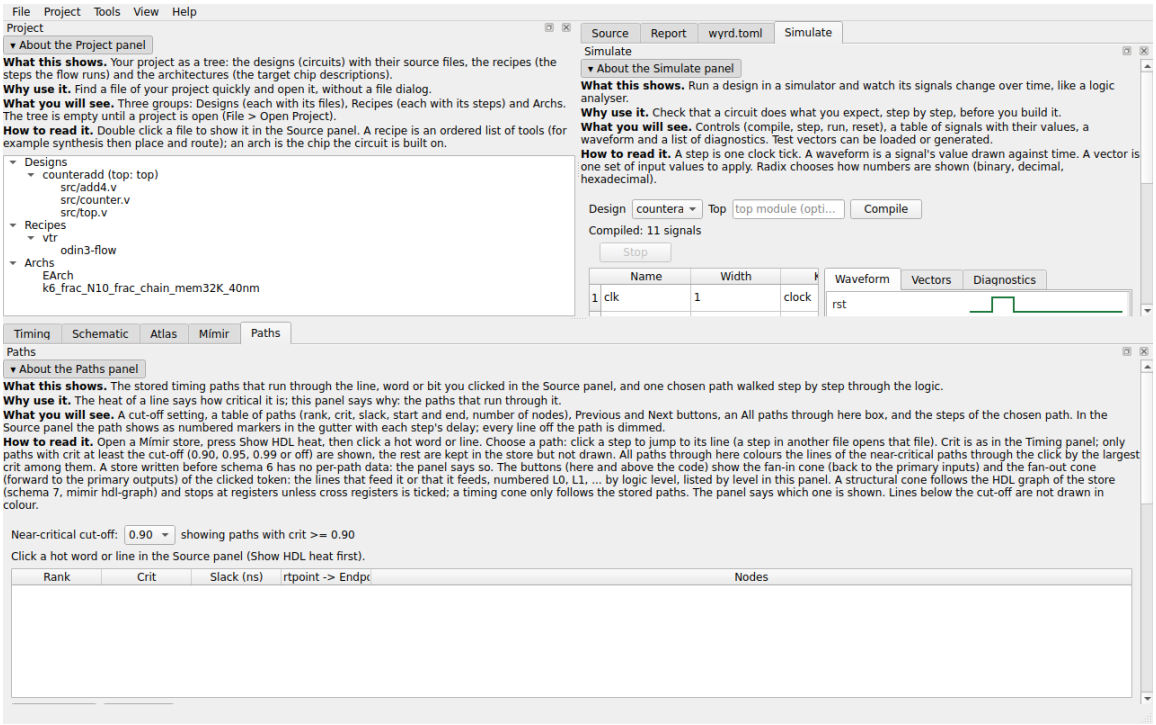


Figure 21. After reset and five cycles with `step` = 9.

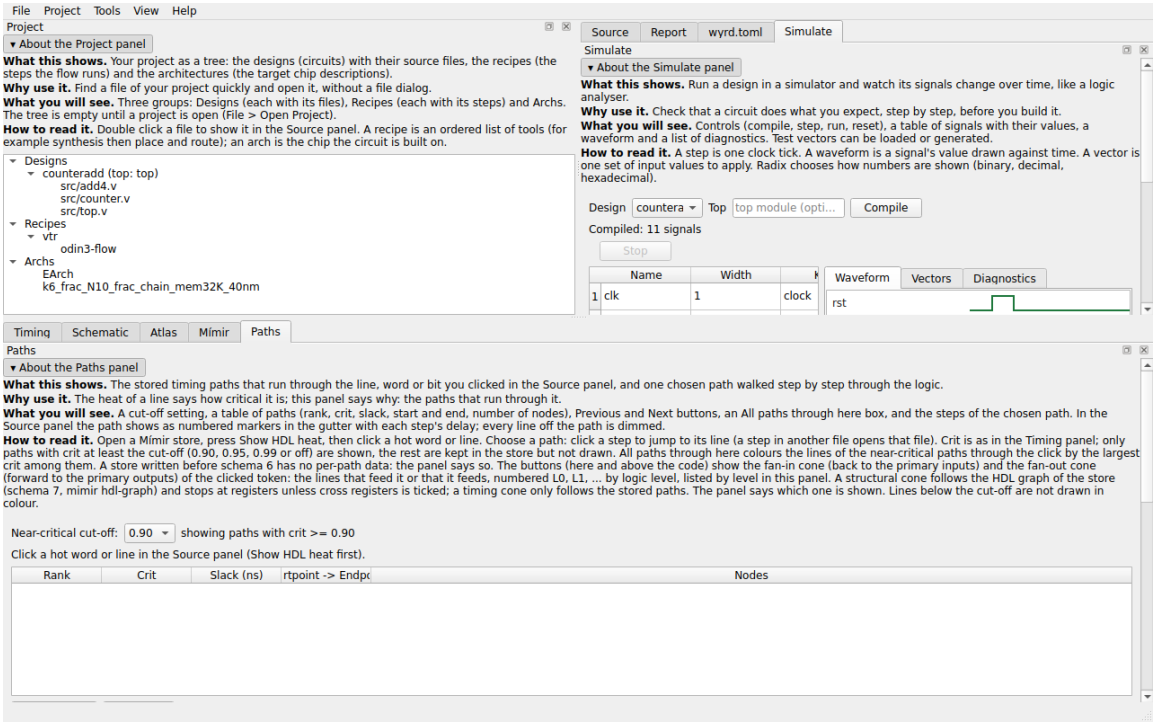


Figure 22. The waveform view after ten more cycles with `step` = 3.

9 The Floorplan

The Floorplan button in the Report tab shows the placement and routing of a finished run in VPR's own graphics window. It needs a VPR built with graphics. A normal VTR build leaves graphics out, so the window would close at once. The steps are: build a graphics VPR once (Section 9.1), tell Wyrd III where it is (Section 9.2), press Floorplan (Section 9.3) and step through the window (Section 9.4).

9.1 Build a VPR with graphics

These steps were checked on Ubuntu 24.04 under WSL2 with VTR 9.0.0-dev. They run in a terminal; `<qt>` and `<dir>` are folders you choose and `$VTR_ROOT` is your VTR source. Nothing needs `sudo`.

1. VTR needs Qt 6.9.3 or newer. Ubuntu 24.04 ships Qt 6.4.2, so install a private Qt with aqtinstall (about 1.7 GB):

```
$ python3 -m venv <qt>/aqt-venv
$ <qt>/aqt-venv/bin/pip install aqtinstall==3.3.0
$ <qt>/aqt-venv/bin/aqt install-qt linux desktop 6.9.3 linux_gcc_64 \
    --outputdir <qt> --modules qtshadertools
```

2. Configure and build only VPR, out of the VTR tree:

```
$ cmake -S $VTR_ROOT -B <dir> -DCMAKE_BUILD_TYPE=Release -DVPR_USE_EZGL=on \
    -DCMAKE_PREFIX_PATH=<qt>/6.9.3/gcc_64
$ cmake --build <dir> --target vpr -j6
```

   The program is `<dir>/vpr/vpr`. Qt is what this VTR needs for graphics. Older VTR builds drew with GTK instead (Ubuntu package `libgtk-3-dev`); with the VTR commit used here the `-DVPR_USE_EZGL=on` configure step stops with "Qt6 ¿= 6.9.3 was not found" until the Qt prefix is given. Versions and install notes for all the tools are on the hub's tools page, https://github.com/drpaj12/YggdrasilIII/blob/main/docs/tools.md. The program finds its Qt libraries through the path recorded in it at build time; if you move the Qt folder, add its `lib` folder to `LD_LIBRARY_PATH`.

3. Check that graphics are in. The command prints `0` when `NO_GRAPHICS` is not defined:

```
$ grep -c NO_GRAPHICS <dir>/vpr/CMakeFiles/vpr.dir/flags.make
0
```

9.2 Point Wyrd III at it

Open Project > Settings... and type the path of the new program in the vpr_gui field of the tools group, for example `<dir>/vpr/vpr`, then press OK (Figure 9, which shows the field filled in). The runs themselves keep using the VPR of vtr_root; only the Floorplan button uses vpr_gui. If the field is empty the button uses the VPR named in the run's log. A path that is not an executable file is refused.

9.3 Open the floorplan

Select a finished run in the tree of the Report tab and press Floorplan, next to the tree. The button is disabled, with the reason as its tooltip, when no VPR can be used. Wyrd III starts VPR again for that

run, with graphics on, and the VPR window opens (under WSL, WSLg shows it). The Flow log prints the command. Messages such as `libEGL warning: failed to get driver name for fd -1` and `MESA: error: ZINK: failed to choose pdev` appear in the log: they mean no GPU driver is visible, drawing uses the CPU, and the window works. Each press of Floorplan opens another VPR window.

9.4 Step through it

The window shows the chip grid. Blocks are drawn in colour with their names (Figure 23). Wyrd III starts VPR with `--graphics_pause final_stage`: VPR packs, places and routes the design first without stopping, then pauses once with the finished picture on screen. The Proceed button at the bottom of the window is what lets VPR carry on and exit. If you want VPR to stop at every stage or every update (placement, then routing), run VPR yourself with `--graphics_pause major` or `every_update`; each pause also waits for Proceed. Under Misc. the switch Proceed by Step and the field Steps to Proceed make VPR pause after a chosen number of updates.

The controls, all named as they are on the buttons of the VPR window:

- Zoom Fit shows the whole chip again; Zoom Select lets you click two points to zoom into the rectangle between them. The mouse wheel also zooms, and dragging with the mouse pans.
- The menu buttons Block, Net, Route and Misc. open the toggles. Block has Block Internals (how deep to draw inside blocks), Block Text, Block Outline, Block Pin Utilization and Placement Macros.
- Net has the switch Display Nets with its type box (Flylines), Inter-cluster Nets and Intra-cluster Nets (Figure 24), and the switch Display Critical Paths with Critical Path Flylines, Critical Path Routing and Critical Path Delays (Figure 25).
- Route has the switch Display Routing Resources with Channel Nodes, Switch Box Edges and Connection Box Edges, and the Congestion and Routing Utilization boxes. They show the routed design.
- Misc. has Proceed by Step, Steps to Proceed, the Save button (writes the picture to a file) and Manual Move.

Close the window when you are done.

Note. A grab of the VPR window is black under WSLg, because VPR draws with OpenGL. The three pictures here were made by VPR itself instead: with `--disp off` VPR draws off-screen, and `--graphics_commands "save_graphics place.png"` writes a PNG. The script `docs/tutorial-gui/shots/floorplan_shots.py` builds the same command as the Floorplan button for the run of the tutorial and saves the pictures: `python floorplan_shots.py $VPR_GUI <run dir> <out dir>`, where `VPR_GUI` is the graphics VPR of this section (the same variable `make_gui_shots.py` puts into its private machine file). They show the same drawing as the window; the Route switches, which only the live window can set, are not in them.

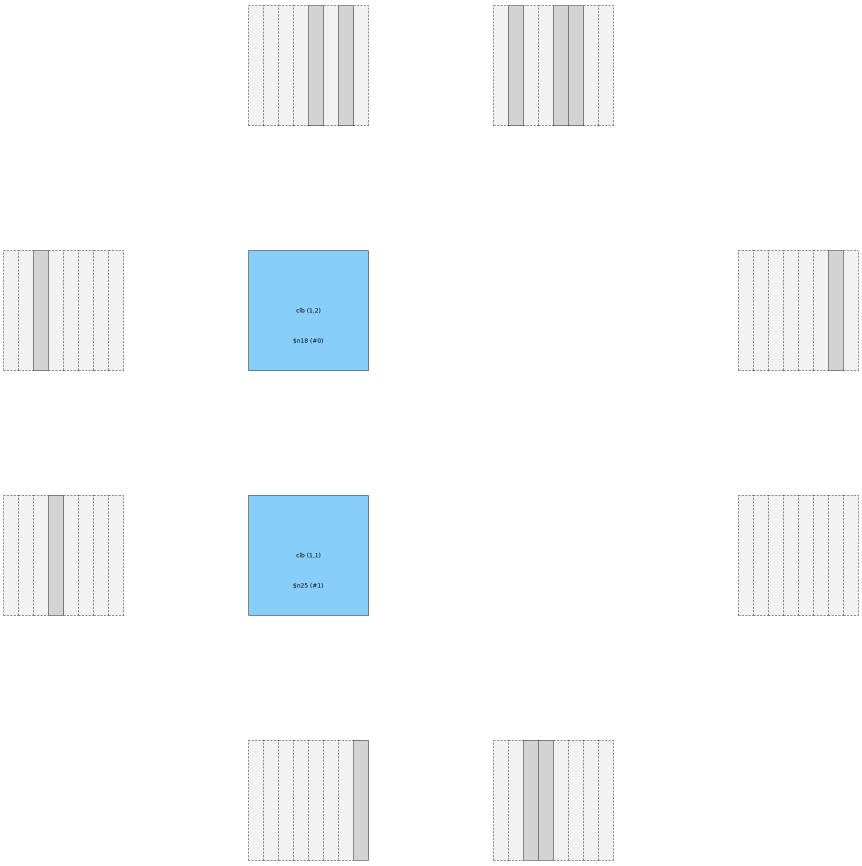


Figure 23. The placement of the tutorial design on the EArch grid: two logic blocks (CLBs) and the I/O pads around them.

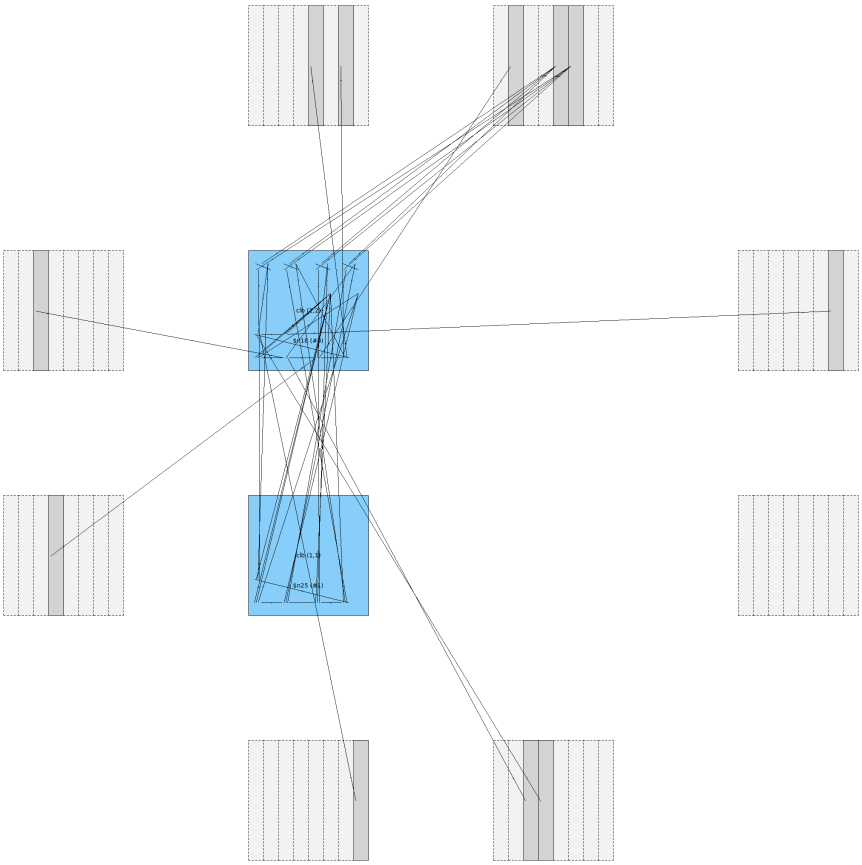


Figure 24. The same placement with the nets drawn as flylines (Display Nets).

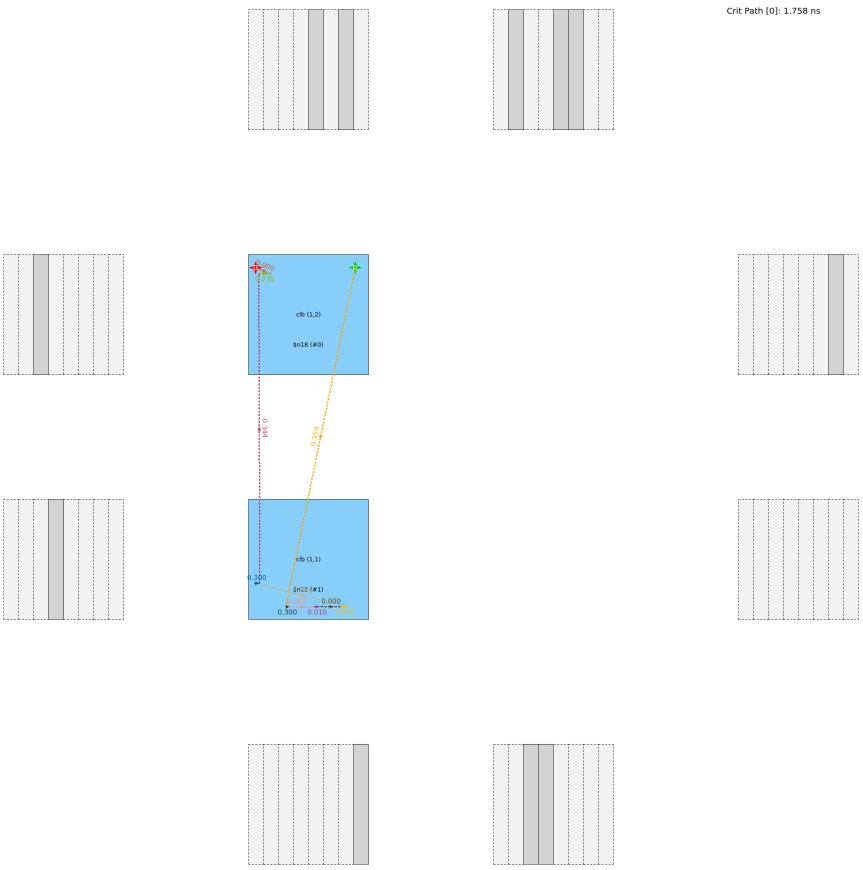


Figure 25. After routing, with the critical path drawn (Display Critical Paths); the heading gives its delay, 1.758 ns.

10 The Sample Project and Midgard III

10.1 The sample project

The repository's `samples` folder is a project with three designs: `single` (a LUT-only design), `multi` (several files in folders) and `dsp` (a memory and a multiplier), run on both architectures. Open it with File > Open Project... (Figure 26), press Run, and select All runs (Figure 27): six runs, `dsp` using one memory and one multiplier block.

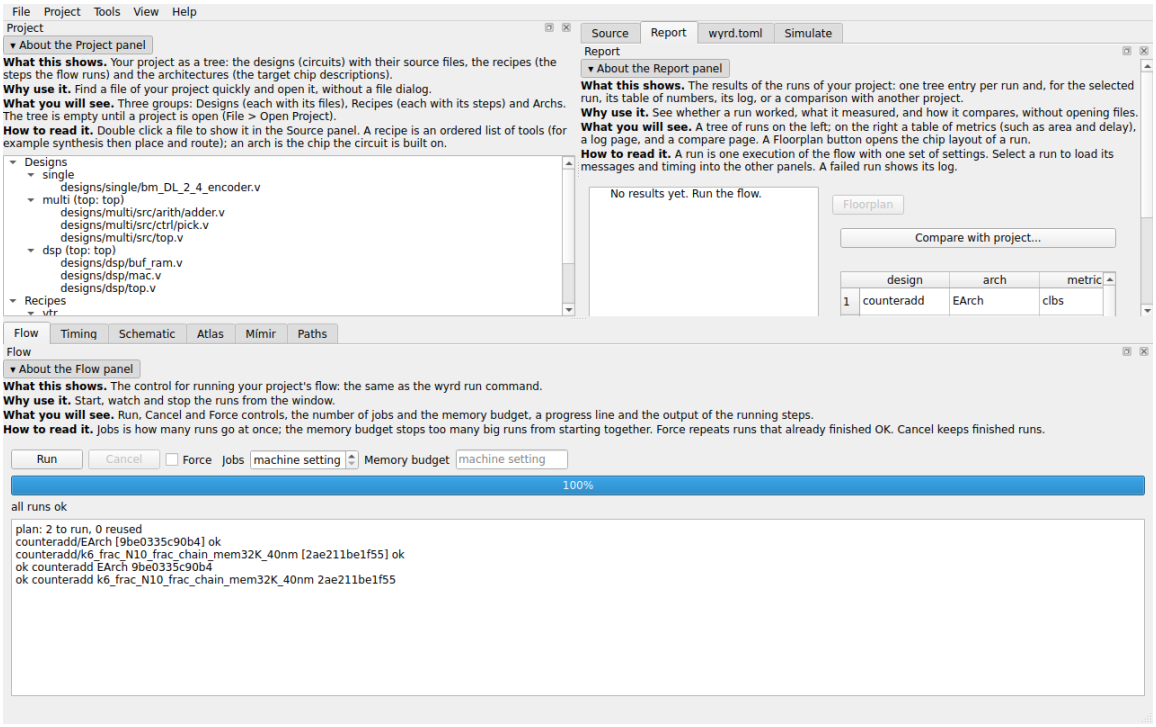


Figure 26. The sample project opened; the tree shows its three designs.

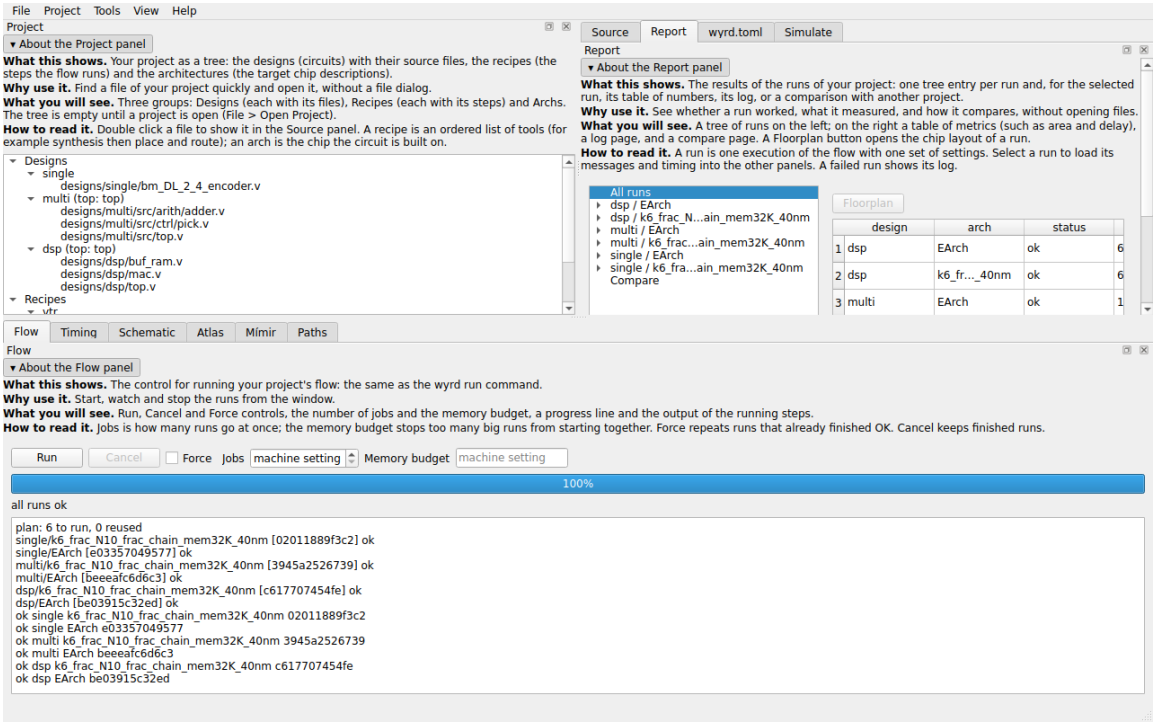


Figure 27. The sample project after Run: six runs.

10.2 Midgard III (Verilog Town)

Midgard III is a game inside Wyrd III, a close replication of verilogTown. You write one Verilog module for each traffic light of a small town, and cars drive through it. The modules are simulated by Odin III through Bifrost III; one tick is one clock edge of your module, 60 ticks a second at speed 1.

Starting the game

The game has three entry points: Tools > Midgard III (Verilog Town) in Wyrd III (Figure 28), the command `midgard` (which also takes `--level N`, 0 to 12, and `--data-dir`), and the level editor, Tools > Midgard III Level Editor or `midgard-editor`. The game needs pygame; without it the first menu entry is greyed with a tooltip that says why.



Figure 28. The Tools menu.

The main menu lists levels 1 to 12, the tutorial level (level 0), the high scores and the credits. Choose level 1 (Figure 29). The game creates the files you own in its data directory (`$MIDGARD_HOME` or `~/.local/share/midgard`): `Lv1/VerilogFiles/Traffic_signal_set_0.v` and one file for each other intersection, from a starter module.



Figure 29. The main menu (left) and level 1 in the edit phase (right). The sensor pile, with the number of sensors left, is at the bottom-left of the map.

The help overlay (H)

Press H in the edit phase or when the level is paused. The help overlay lists the keys (Figure 30); press H again to close it. The overlay shows S (start), P (pause), M (main menu), R (restart), = and - (zoom in and out), the four arrow keys (move the map), C (clear sensors) and H (help). The game also reacts to the keys listed in Table 1, which the overlay does not show.



Figure 30. The help overlay of Midgard III.

Key	Action
S	start the level (edit phase), or resume (paused)
P or X	pause (running)
. (period)	step one tick (starts paused in the edit phase)
1, 2, 4	speed 1x, 2x, 4x
V	arm or stop recording
E	export a VCD of each recorded intersection (paused or finished)
R	restart the level
M	main menu (edit phase)
C	remove all sensors (edit phase)
H	help overlay (edit phase or paused)
Q or =; A or -	zoom in; zoom out
arrow keys	move the map

Table 1. The keys of the level screen, as the code binds them.

Placing and dragging a sensor

A level has seven general sensors, numbered 0 to 6. A sensor on a road tile tells your Verilog whether a car is on that tile and where it is going. Sensors can only be placed in the edit phase.

1. Click the sensor pile at the bottom-left of the map: the highest-numbered unplaced sensor follows the mouse (Figure 31). Click a road tile that is not part of an intersection to place it (Figure 31). The sensor file is saved at once.

2. To move it, press on the placed sensor, drag it to another road tile and release (Figure 32). Right-click removes a sensor; C removes all.



Figure 31. Sensor 6 in the hand, and placed on a road tile.



Figure 32. Dragging a placed sensor (left) and after releasing it on a new road tile (right).

Writing the Verilog of a light

Click an intersection: the Verilog editor opens with the file of that light (Figure 33). The module `light1` has the ports `clk`, `rst` (active low), `sensor_light` (8 bits: the entries and the inside of this intersection), `general_sensors` (30 bits: the sensors you placed), the four outputs `outN`, `outS`, `outE` and `outW` (3 bits: 0 Stop, 1 Forward only, 2 Left only, 3 Right only, 4 Go) and an optional `debug_port`. Here the starter has one change, `assign debug_port = general_sensors;`, so the game shows what the sensors deliver. Press Verify to compile the file with Odin III (Figure 34). Tick a box in the picture of the intersection or type bits in Internal Sensors and press Simulate to step the module one cycle and print its outputs (Figure 35): with a car on the north entry the light answers `OutN = 4`, Go. You can also edit the file in any editor; the game notices the change and recompiles it in the edit phase.

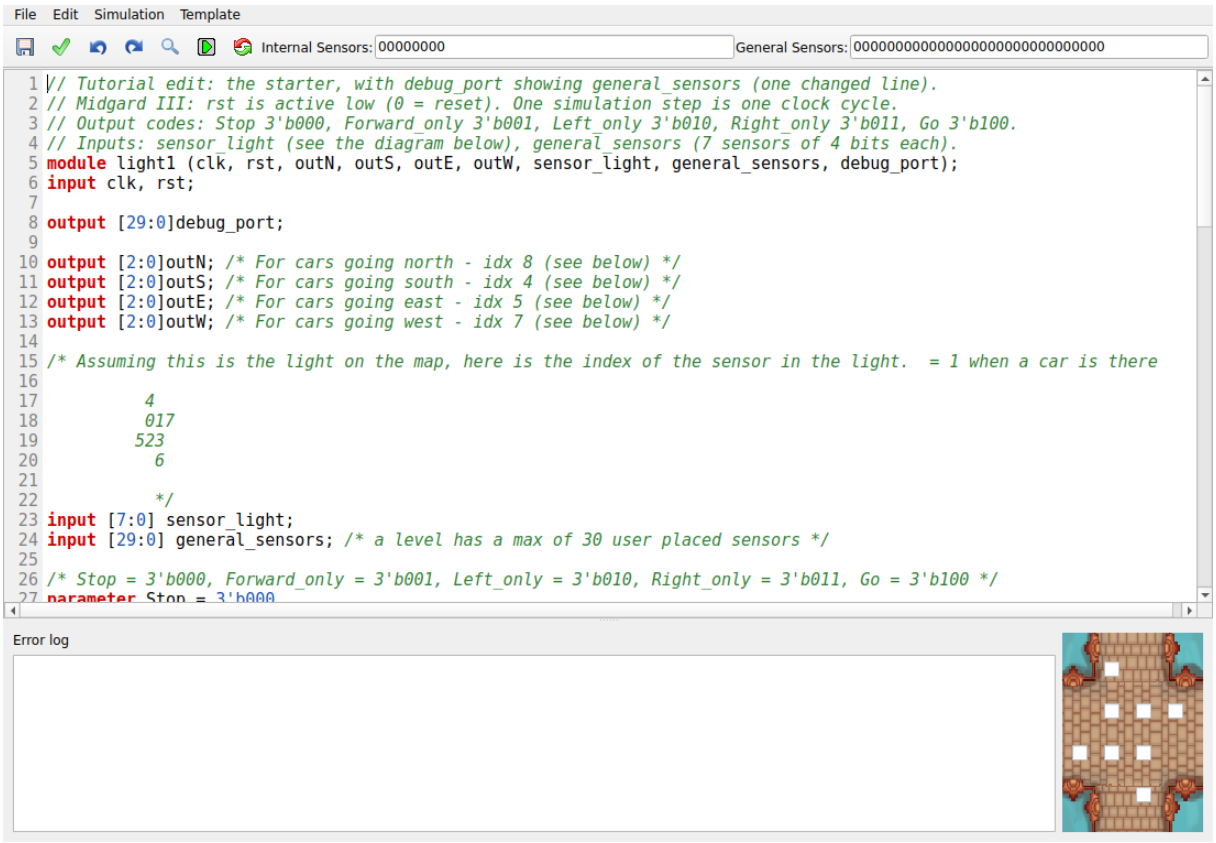


Figure 33. The Verilog editor with the file of the first light.

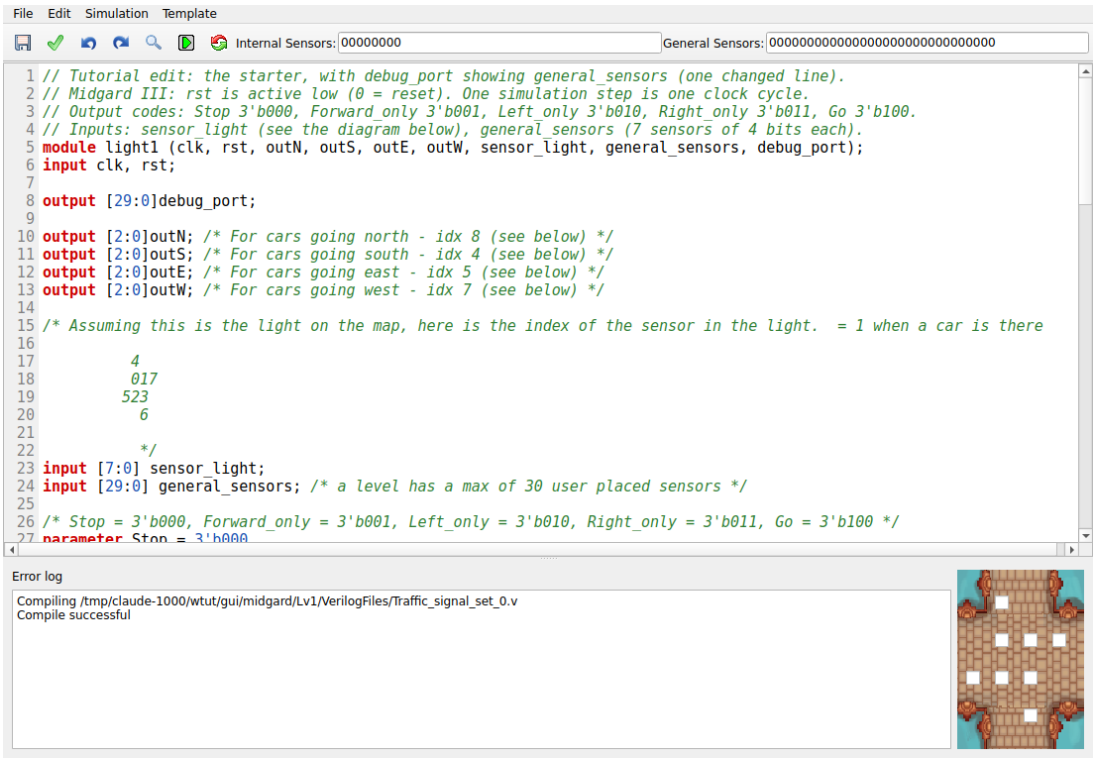


Figure 34. After Verify.

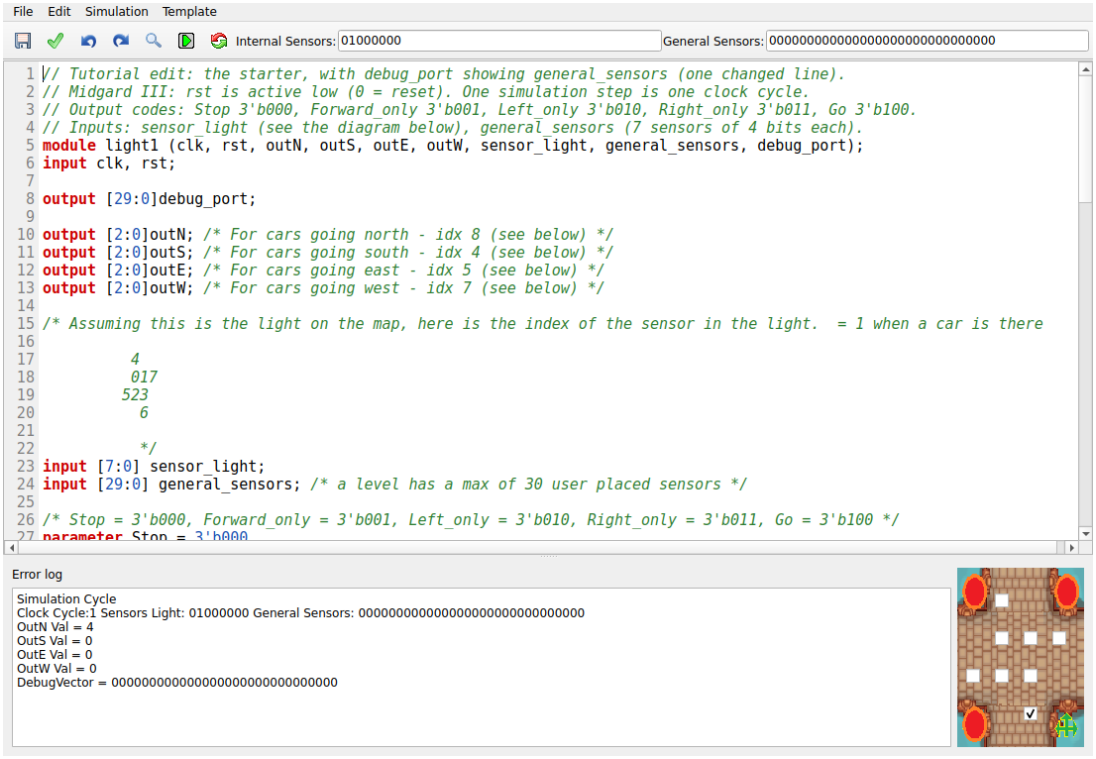


Figure 35. After Simulate with a car on the north entry.

Starting the simulation (S)

Close the editor and press S. The cars drive; the status line shows the speed and the tick rate, and the buttons under the map change the speed, step, record, export a VCD and open a waveform. Press P to pause when a car is on your sensor: the strip under the map then lists, for every intersection, the inputs of the last tick and its four outputs (Figure 36). Press S to resume. When all cars are done the score screen shows the cars that passed and the time in ticks (Figure 36); the ten best results of each level are kept.

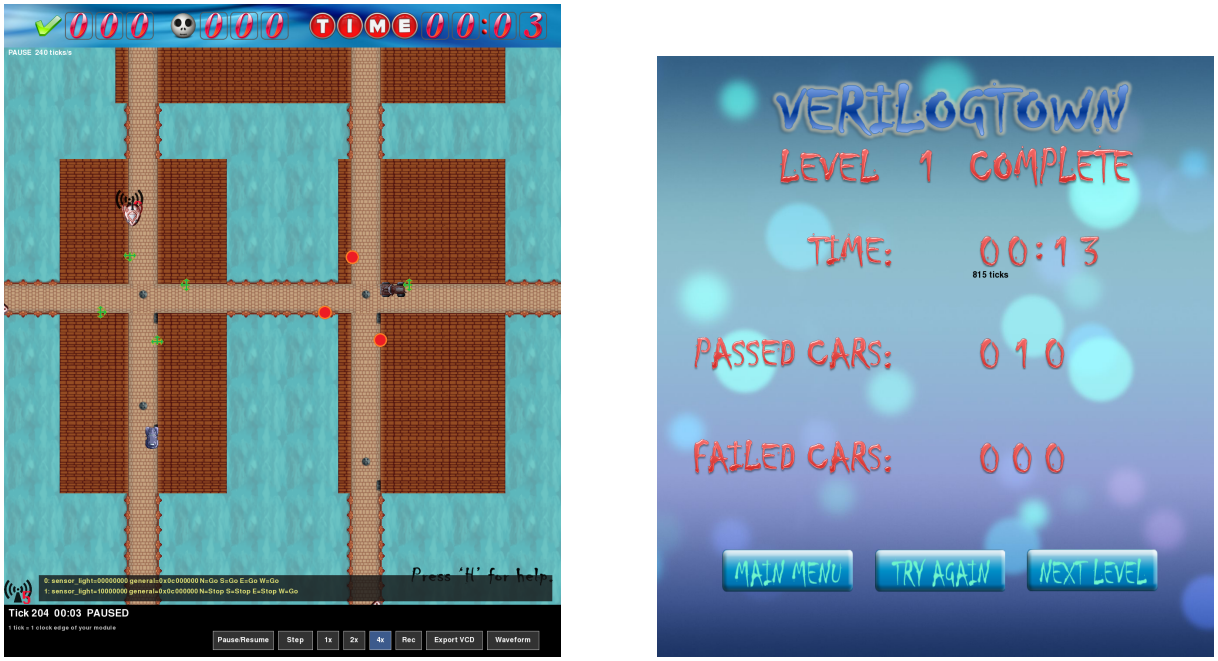


Figure 36. Level 1 paused with a car on the sensor (left) and the score screen (right).

The level editor

Tools > Midgard III Level Editor opens a window for making your own levels (Figures 37 and 38); it needs no pygame.

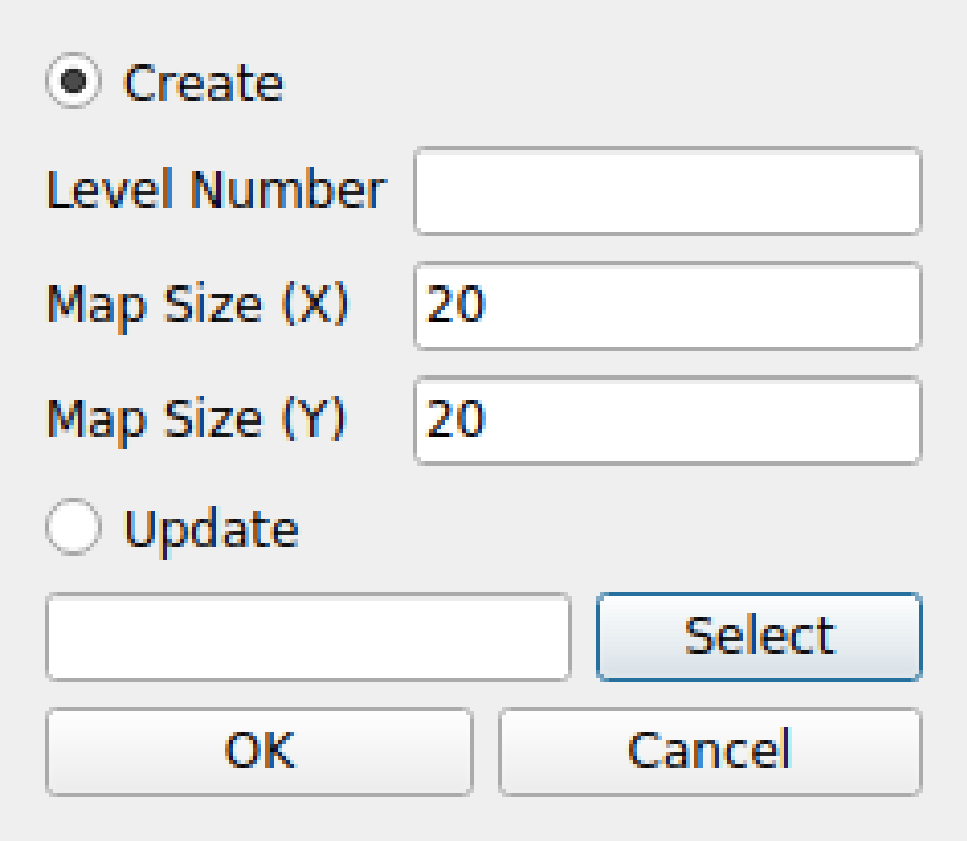


Figure 37. The level editor start window.



Figure 38. The level editor with level 1 loaded.

11 Where to Go Next

- The Wyrd III manual: https://drpaj12.github.io/wyrd/, with every setting, option and key.
- The command-line tutorial: https://drpaj12.github.io/wyrd/wyrd-tutorial.pdf, the same project run from a terminal, with the run directory, `wyrd table`, `wyrd compare` and `wyrd sim`.
- The Yggdrasil III hub, with the other tools of the suite (Odin III, Loki III) and how they fit together: https://github.com/drpaj12/YggdrasilIII.